

Shorts 1: Pytest

If the function being tested has its own case of error checking and we want to test if the error checking works:

```
ex. def test_error():
    with pytest.raises(<ErrorType>):
        funcname(<ErrorValue>)
```

Here we are testing "funcname" function's error by making the function "test_error". "funcname" gets some value that would error as an argument, and with that argument passed, pytest would raise the ErrorType expected. Pytest will still pass.

In many forms of testing floats are not easily represented because there are an infinite number of floating point values. Some values can be very large ($1.000000001 \sim$), so approximation is necessary.

```
ex. Assume funcname does math
def test_float():
    assert funcname(0.001) == pytest.approx(12.34)
```

The .approx method can change the tolerance depending on user.

```
ex. def test_float():
    assert funcname(0.001) == pytest.approx(12.34, abs=0.1)
```

The test will accept values 12.34 ± 0.1

Tolerance can be better adjusted using $abs = 1e-n$